Final Report

Level 4

A Multi-Signal, Co-Scheduling Autoscaling Framework with Adaptive Control for SLA-Driven Kubernetes Workloads

Promex

214030K	Bandara K.G.R.U.
214060C	Diwyanjalee E.A.D.S.N.
214129X	Malalpola M.L.H.R.

Faculty of Information Technology

University of Moratuwa

2026

Final Report

Level 4

A Multi-Signal, Co-Scheduling Autoscaling Framework with Adaptive Control for SLA-Driven Kubernetes Workloads

Promex

214030K	Bandara K.G.R.U.
214060C	Diwyanjalee E.A.D.S.N.
214129X	Malalpola M.L.H.R.

Supervised by: Ms. M.N. Chandimali

Faculty of Information Technology

University of Moratuwa

2026

Abstract

Applications deployed on Kubernetes are expected to honour service level agreements while keeping infrastructure cost under control, yet the platform's standard autoscaling behaviour falls short of this expectation in three specific ways. Scaling decisions react to a single, lagging resource metric; the scheduler that places newly created replicas knows nothing about why they were created; and the numeric settings that govern scaling remain fixed after deployment even as workload behaviour drifts. This project designed, built, and evaluated an integrated framework, named Promex, that addresses the three weaknesses together rather than in isolation. The framework serves platform operators who run latency-sensitive services. Its inputs are ordinary runtime measurements: processor and memory utilisation, request arrival rates, and latency percentiles. Its outputs are two coordinated decisions, namely how many replicas to run and where each new replica should be placed. Three cooperating modules produce these decisions. A gradient-boosted classifier fuses the incoming measurements into a single violation-risk score and attributes each score to the measurement responsible for it. A discounted Thompson Sampling policy, exposed to the cluster as a scheduler extender, uses that attribution together with live node conditions to choose a node for every new replica. A proportional-integral controller, bounded by conformally calibrated uncertainty and by a guard that reacts to the controller's own recent reversals, continuously adjusts the alert threshold that drives scaling. The modules were first trained and validated offline against a twelve-hour production trace of a large microservice platform, then deployed on a two-node Kubernetes cluster running the TeaStore reference application under replayed traffic. A twenty-five-trial controlled comparison across five system configurations showed that adaptive control driven by a raw processor signal over-provisioned the service for 57.8 percent of each run on average, while the same controller driven by the fused risk score reduced this to 2.2 percent, at the lowest cost and with the fewest violations of any configuration tested.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Introduction

Container orchestration has become the ordinary way to run online services at scale, and Kubernetes is the platform most organisations reach for. Once a service is deployed there, two questions must be answered continuously: how many copies of it should be running, and which machine should each copy sit on. This report describes a framework, named Promex, that answers both questions together instead of separately. The chapter that follows sets out why the separation matters, states what the project set out to achieve, sketches the solution, and explains how the rest of the report is organised.

1.2 Background and Motivation

A service level agreement puts a number on acceptable performance, and breaching that number carries a commercial cost. Autoscaling is the mechanism through which platforms try to honour such agreements without paying for idle capacity. Kubernetes ships with several autoscalers for this purpose. The Horizontal Pod Autoscaler adjusts replica counts against a target metric, the Vertical Pod Autoscaler resizes individual containers, the Cluster Autoscaler adds and removes machines, and event-driven extensions allow scaling against external queues. Each of these operates on numeric settings that an operator chooses once, at deployment time, and rarely revisits.

Three weaknesses in this arrangement recur across the published literature. The first concerns which measurement drives the decision. Processor utilisation remains the default trigger, yet it is a trailing indicator: it climbs after queues have already built and after users have already waited. Work on application-level observability has shown that a queue-depth signal tied directly to a stated agreement can cut violations dramatically compared with a processor-only trigger [6]. Even so, that approach and others like it read one measurement at a time rather than combining several into a single view of risk.

The second weakness is a division of labour inside the platform itself. An autoscaler decides that another replica is needed; the scheduler then decides where that replica lands, using its own scoring rules and without any knowledge of what prompted the request. Vu, Tran and Kim built a hybrid autoscaler that combined horizontal and

vertical adjustment with a demand forecaster, and reduced violations during traffic bursts from roughly twenty per cent to under four per cent, but their published account states plainly that coordinating the scaling decision with placement was left outside the study [10]. The gap is therefore acknowledged rather than closed.

The third weakness is that control settings do not move. Thresholds, step sizes and inspection intervals are fixed when the service is deployed. Because workload behaviour drifts, a threshold that was sensible in one week becomes either too eager or too slow in the next, and the familiar result is either wasted capacity or an oscillating replica count. This is not a problem confined to older systems. A recent proactive framework built on a deep Q-network with a demand forecaster still uses a fixed thirty-second decision interval and a fixed action set of one replica up, one replica down, or hold; its authors identify widening that action set as outstanding work [11]. Their own reported figures are instructive: the system ran the highest average replica count of the three compared, consumed the most resource, and produced twice as many scaling events as either baseline.

Taken singly, each weakness degrades either agreement compliance or cost. Taken together they compound, because a correctly detected need for capacity can still be squandered by poor placement, and a well-placed replica can still arrive too late or too often when the component that requested it cannot adjust its own behaviour. No framework identified during the review of existing work brings signal fusion, placement awareness and self-adjusting control into one system and measures what each part contributes.

1.3 Aim and Objectives

Aim. To design, build and evaluate an integrated autoscaling framework for Kubernetes that fuses several runtime measurements into one agreement-aware risk score, couples replica placement to the cause of each scaling decision, and adjusts its own control settings as workload behaviour changes, then to measure the effect of doing so against the platform's standard autoscaler.

Objectives.

1. To build a model that combines backlog, processor utilisation and latency-percentile measurements into a single risk score, and to expose which measurement is responsible for each score it produces.

2. To build a placement mechanism that selects a node for every new replica using the identified cause of the scaling decision together with live node conditions.

3. To build a control mechanism that adjusts its own alert threshold within bounds derived from its recent prediction error and its own recent instability.

4. To evaluate the three parts separately and in combination against the platform's standard autoscaler on a running cluster, and to report what each part contributes to agreement compliance, cost and stability.

1.4 The Proposed Solution

Promex is intended for the engineers who operate latency-sensitive services on Kubernetes and who currently tune autoscaler settings by hand. It reads measurements the platform already collects, namely processor and memory utilisation for the target service, its inbound request rate, and response-time percentiles computed from call records. From these it produces two coordinated outputs: a replica count for the service, and a chosen node for each replica the platform creates.

Between input and output sit three cooperating parts. A gradient-boosted classifier turns the measurement vector into a probability that the service will breach its latency target during the next interval, and reports which measurement family drove that probability. A discounted Thompson Sampling policy, presented to the cluster as a scheduler extender, treats each candidate node as an option whose value it learns from what happened after previous placements. A proportional-integral controller moves the alert threshold that governs scaling, with the size of each move bounded by conformally calibrated prediction error and further restrained whenever the controller has recently reversed direction.

The implementation runs on Python with gradient boosting, attribution and scientific-computing libraries, packaged into containers and deployed onto a Kubernetes cluster built with Kubernetes in Docker. The reference application under test is TeaStore. Traffic is generated inside the cluster by k6. Modest hardware is sufficient: the entire evaluation was carried out on a single workstation hosting a two-node cluster.

1.5 Structure of the Report

Chapter 2 examines how others have approached elastic scaling on container platforms and identifies what their work leaves unresolved. Chapter 3 explains the specific techniques adopted here and argues why each suits the problem it was chosen for, including the reasoning behind rejecting deep reinforcement learning. Chapter 4 describes the approach in terms of its users, inputs, processing and outputs. Chapter 5 presents the analysis and design, beginning with the top-level architecture. Chapter 6 covers implementation, including the hardware and software environment and the algorithms used in each part. Chapter 7 reports the evaluation and compares the outcome with related systems. Chapter 8 concludes the report, stating the limitations honestly and setting out what should follow. Supporting material appears in the appendices.

1.6 Summary

This chapter has established that standard autoscaling on Kubernetes reacts to a trailing signal, places new replicas without regard to why they were requested, and operates on settings that never move. It stated the aim of addressing all three together, listed four objectives, and outlined the framework built to meet them. The next chapter turns to the published work these weaknesses were drawn from.

Chapter 2

Elastic Scaling on Container Platforms

2.1 Introduction

The previous chapter asserted three weaknesses in current autoscaling practice. This chapter supports those assertions from the literature. It begins with the mechanisms Kubernetes provides and the taxonomy the field uses to classify scaling techniques, then works through the three areas in turn, compares the closest systems in a single table, and closes by stating the gaps this project addresses.

2.2 Standard Mechanisms and How the Field Classifies Them

Lorido-Botran, Miguel-Alonso and Lozano surveyed elastic scaling for cloud applications and sorted the techniques into five families: static threshold rules, control theory, reinforcement learning, queueing theory and time-series analysis [3]. The classification remains serviceable. Production autoscalers on Kubernetes belong almost entirely to the first family, while research has moved towards the remaining four, frequently in combination.

Empirical evaluation of the platform's own components is less common than one might expect. Tamiru, Tordsson, Elmroth and Pierre compared Cluster Autoscaler configurations on a managed cluster using the TeaStore application, and found that scaling behaviour depends chiefly on the composition of the workload rather than on autoscaler configuration alone [9]. Their study also supplies the elasticity measures adopted later in this report: the share of time a service spends over-provisioned or under-provisioned, the instability of the supply curve, and its deviation from an ideal trajectory. Reproducibility of such comparisons is itself a recognised difficulty. Xie and colleagues released an automated test bed precisely because setting up consistent autoscaler comparisons by hand is laborious and error-prone [12]; their published configuration uses a single worker node, which makes placement effects impossible to observe.

2.3 Which Measurement Should Drive Scaling

Evidence that moving beyond processor utilisation improves outcomes is consistent. Patharlagadda mapped queue backlog directly to replica counts, defining compliance

in terms of backlog limits rather than an inferred proxy, and reported violation reductions of up to eighty-six per cent against processor-based scaling together with faster reaction and less replica churn [6]. The approach is deterministic and reads a single measurement, which leaves open how several complementary measurements might be combined.

Vu, Tran and Kim went further on the modelling side, pairing a bidirectional recurrent forecaster with a burst detector and mixing horizontal with vertical adjustment [10]. Violations during bursts fell from around twenty-one per cent under reactive scaling to under four per cent, with lower normalised cost than the proactive baselines they reimplemented. Their burst thresholds and inspection interval are nonetheless fixed, and placement is explicitly out of scope.

Punniyamoorthy and colleagues come closest to a fused approach. Their framework prioritises stated objectives over raw utilisation, deliberately keeps its decision logic explainable, and reports cumulative violation duration rather than a simple count on the grounds that duration reflects user impact more faithfully [7]. Across bursty, queue-driven and mixed workloads they measured up to thirty-one per cent lower cumulative violation duration and eighteen per cent lower cost than the untuned standard autoscaler. Their stabilisation windows and rate limits remain fixed, and the node-capacity signal they emit is a single hint rather than a placement score.

2.4 Placement and Its Separation from Scaling

The Kubernetes scheduler filters and scores candidate nodes on resource availability, affinity rules and topology constraints. What it never receives is the reason a replica was created. This is structural rather than accidental: the autoscaler and the scheduler are separate control loops whose only shared vocabulary is a replica count. Because assigning containers to machines optimally is a combinatorial problem that becomes intractable at scale, schedulers of every kind rely on heuristic scoring instead of exact assignment, which leaves room for a scoring function that takes the scaling cause into account. In the reviewed literature the coupling is treated as an acknowledged omission rather than a contested design question, and the omission recurs even in otherwise sophisticated systems [10].

2.5 Self-Adjusting Control

The reference model for systems that manage themselves remains the monitor-analyse-plan-execute loop over shared knowledge proposed by Kephart and Chess, which frames adaptation as continuous rather than as a configuration exercise [1]. Within the taxonomy noted earlier, the control-theoretic and reinforcement-learning families both offer routes to adaptation, and the literature does not settle between them.

On one side, Pandey integrated a policy-gradient agent with a multidimensional autoscaler on a nine-node managed cluster and reported thirty per cent higher processor utilisation, fifteen to twenty per cent lower ninetieth-percentile latency, and roughly twenty per cent lower cost than the standard autoscalers, with significance established across twenty runs [5]. On the other, Qiu and colleagues, working on fine-grained resource management for services with stated objectives, document that learned resource-management policies tend to be specific to the workload and infrastructure they were trained on and require retraining as conditions move [8]. Neither position invalidates the other; the difference appears to lie in how far the evaluation environment resembles sustained production.

The most recent evidence sharpens the point. Wanigasooriya and Ekanayake combined a duelling deep Q-network with a recurrent forecaster and a language-model validation layer, an architecturally ambitious design [11]. Their decision interval is fixed at thirty seconds and their action set at three choices. Read against their own results table, the system produced the highest average replica count of the three compared, the largest resource consumption, and eight scaling events against four for each baseline. Fixed control settings therefore persist in current work, and prediction alone does not deliver stability.

A directly relevant control-theoretic design comes from Liu and colleagues, who wrapped a forecaster in adaptive conformal inference to calibrate its uncertainty and used a proportional-integral controller to pace provisioning against a violation budget on a Kubernetes controller [2]. That pairing is the starting point for the control mechanism described in Chapter 3, and the distinction between their design and the one built here is stated explicitly in Section 7.8.

2.6 Comparison of the Closest Approaches

Table 2.1 sets the reviewed systems against the three areas of interest. The pattern is that each addresses one or two, and none addresses all three within a single evaluated system.

Table 2.1: Coverage of the three areas of interest by the closest reviewed systems

Reviewed work	Fused measurements	Placement coupled to scaling	Self-adjusting settings
Patharlagadda [6]	Single application measurement	Not addressed	Fixed deterministic mapping
Vu, Tran and Kim [10]	Forecast of one demand series	Excluded by the authors	Fixed burst thresholds and interval
Punniyamoorthy et al. [7]	Several measurements, objective-first	Binary capacity hint only	Fixed stabilisation windows
Pandey [5]	Not a fusion design	Not addressed	Learned policy, retraining cost
Wanigasooriya and Ekanayake [11]	Forecast of one measurement	Not addressed	Fixed interval and action set
Liu et al. [2]	Single forecast series	Not addressed	Calibrated pacing, no self-reference
Promex (this project)	Three families fused with attribution	Attribution feeds a node score	Bounded self-adjustment with an oscillation guard

2.7 Gaps Addressed by This Project

Four statements follow from the review. Backlog, processor utilisation and latency percentiles are not combined into one validated risk score anywhere in the reviewed work. Placement is not coupled to the cause of a scaling decision; the nearest systems either exclude it or emit an unscored hint. Fixed thresholds, step sizes and intervals survive into current proactive designs by their authors' own admission. And no single system integrates solutions to all three while measuring what each contributes, which is the omission this project treats as its central concern.

Two tensions in the literature deserve to be named rather than resolved prematurely. Learned control reports strong benchmark gains [5] alongside documented fragility once conditions drift [8], a divergence that appears to track the realism of the evaluation setting. Separately, adding measurements improves discrimination but also multiplies the number of inputs that must remain trustworthy, which is a cost this project acknowledges in Section 8.3 rather than claims to have eliminated.

2.8 Summary

The reviewed work converges on three specific shortcomings and one structural absence. Individual mechanisms for better signals, better placement and adaptive control all exist; an integrated system that quantifies the contribution of each does not. Chapter 3 sets out the techniques selected to build such a system and the reasoning behind each choice.

Chapter 3

Techniques Selected for Fusion, Placement and Control

3.1 Introduction

Three separate decisions had to be made: how to turn a vector of measurements into a risk estimate, how to choose among candidate nodes, and how to adjust a control setting safely. This chapter explains what was adopted in each case and why the choice fits the problem, then addresses the question of deep reinforcement learning directly, since that was the project's initial expectation.

3.2 Gradient-Boosted Trees with Per-Decision Attribution

Predicting whether a latency target will be breached during the next interval, given the current measurements, is a supervised classification problem with labels available from the data itself. Gradient-boosted decision trees suit it for three practical reasons. They train in seconds on a processor without specialised hardware, which mattered given the workstation used. They handle the mixture of levels and rates of change present in the feature set without scaling or distributional assumptions. And their structure permits exact attribution of a prediction to its input features at low cost.

That last property is the reason the technique was chosen over a small neural network. A fused score tells an operator that risk is elevated; it does not say which measurement is responsible. Tree-based attribution closes that gap and produces, for every prediction, a ranking of measurement families by their contribution. The placement mechanism then consumes that ranking as context. Explainability is not decoration here: the objective-first framework reviewed earlier treats it as a design constraint for exactly this reason [7], and a learned score whose reasoning cannot be inspected is difficult to operate.

3.3 Contextual Bandits with Discounted Updates

Selecting a node is a repeated choice among options where the outcome of the chosen option is observed and the outcomes of the alternatives are not. This is the setting contextual bandit algorithms were designed for, and Thompson Sampling is among the better-understood members of that family. Each node carries a beta-distributed belief

about its quality; a sample is drawn from every belief and the highest sample wins; the belief for the chosen node is then updated from the observed result.

Standard Thompson Sampling assumes each option's quality is stationary. Cluster nodes are not stationary. Other workloads arrive and depart, and machines become busier or quieter across the span of an experiment. The adaptation used here applies a discount factor to the accumulated belief before each update, so that recent evidence carries more weight than stale evidence. The change is small, confined to the update step, and does not alter the algorithm's structure. Its justification is that the closest published use of bandits for placement on this platform concerns a one-off assignment made at cluster construction, where drift does not arise, whereas the decision addressed here recurs throughout the life of the service.

A deliberately plain heuristic accompanies the policy: score each candidate by the product of its available processor and memory headroom and take the highest. It serves as the cold-start behaviour before enough evidence has accumulated, and as a baseline the learned policy must beat. Section 7.4 reports that in one respect it did not.

3.4 Proportional-Integral Control with Calibrated Bounds

Moving a threshold automatically raises an obvious hazard: an adjustment mechanism can itself become a source of oscillation. Classical feedback control addresses this directly and supports reasoning about stability, which learned policies do not. A proportional-integral controller was therefore adopted, acting on the error between observed risk and its target, with an anti-windup provision that prevents integral accumulation while the output sits at a bound and the current error would push it further past that bound.

How far the controller may move at each step is not fixed. Adaptive conformal inference maintains a running quantile of the risk model's own prediction error and supplies an interval that reflects how well the model is currently performing. When prediction quality falls, the interval widens and the controller becomes more conservative. This pairing of calibrated uncertainty with proportional-integral pacing follows the design published by Liu and colleagues [2].

One element goes beyond that design. The published mechanism calibrates from forecast error alone; it holds no view of the controller's own recent conduct. The version

built here counts direction reversals in the controller's recent output and inflates the calibrated interval in proportion to that count, so the permitted step shrinks precisely when instability is already present rather than only when the input signal is noisy. The honest qualification is that inflating a conformal interval in this way trades strict coverage guarantees for a practical safety margin; the adaptive variant already relaxes the exchangeability assumption for related reasons, so the departure is a matter of degree.

3.5 Why Deep Reinforcement Learning Was Not Adopted

The project began expecting to use deep reinforcement learning throughout, and the decision to move away from it was made for different reasons in each of the three parts. For risk estimation the mismatch is structural. No action alters the next state and no reward accumulates over a trajectory; the label is already present in the data. Casting supervised classification as a sequential decision problem would require inventing an action space and a reward, and would obtain through much greater sample cost what labels supply directly.

For placement the mismatch is one of scale. Credit assignment over long horizons is what value-function machinery exists to solve, and node selection does not require it: the consequence of a placement resolves within the same episode. Bandit methods are far more sample-efficient precisely because they omit that machinery. There is also a positioning argument. Deep learned schedulers for this platform are numerous, so adopting one would place the work inside a crowded field rather than distinguish it.

For control the technique fits the problem shape, and here the decision was a deliberate trade rather than a mismatch. Two considerations settled it. A bounded classical controller supports an analytical stability argument, whereas a learned policy can only be observed to behave well. And the evidence assembled in Chapter 2 counts against learned control for a component whose entire purpose is stability: the deep Q-network design was the least stable of the three systems in its own comparison [11], and learned policies are documented to require retraining as conditions drift [8]. A learned variant remains a reasonable extension, and Section 8.4 records it as such.

3.6 Supporting Platform Technology

Beyond the three techniques, the work rests on ordinary platform components. Kubernetes in Docker provides a multi-node cluster on a single host at low memory cost, which the available hardware made necessary. The scheduler extender interface allows a custom scoring service to influence placement without replacing or rebuilding the scheduler. TeaStore serves as the application under test because it is an established reference for this kind of measurement [9]. Load is generated by k6, chosen over alternatives because it ships as a single binary with no coordinator process to accommodate, and because its staged model maps cleanly onto a time-varying request-rate curve. Trace data comes from a published production release covering roughly twenty thousand services over twelve hours [4].

3.7 Summary

Gradient boosting with attribution was adopted for risk estimation because the problem is supervised and the attribution is needed downstream. Discounted Thompson Sampling was adopted for placement because the decision is single-step and the environment drifts. Proportional-integral control with conformally calibrated bounds and an oscillation guard was adopted for adaptation because stability can be argued rather than merely observed. Chapter 4 describes how these fit together into a working approach.

Chapter 4

The Promex Approach to Coordinated Scaling

4.1 Introduction

Chapter 3 justified three techniques in isolation. This chapter describes how they were assembled into a single approach, presented in the terms an operator would care about: who uses it, what it consumes, what it does with that input, what it emits, and what it needs from the environment in order to run.

4.2 Users and Operating Context

The intended user is a platform engineer responsible for a latency-sensitive service running on Kubernetes, who today sets autoscaler thresholds by hand and adjusts them reactively when something goes wrong. Promex is not designed to displace that person's judgement. It is designed to remove the recurring manual tuning, and to make each automated decision inspectable so that trust can be built incrementally.

The framework was built to sit alongside the platform's existing components rather than replace them. The standard autoscaler continues to function; the placement component attaches to the scheduler through its extender interface instead of substituting a new scheduler; and the risk model runs as an ordinary workload that exposes an endpoint. An operator can therefore adopt one part, observe it, and adopt the next, which matters because well-performing research autoscalers rarely become production defaults and reluctance to hand over control is a large part of the reason.

4.3 Inputs

Nothing exotic is required. For the target service the framework reads processor and memory utilisation, the rate of inbound requests, and response-time percentiles at the ninety-fifth and ninety-ninth levels. It also reads per-node processor and memory utilisation for every candidate machine. Alongside each measurement it computes rates of change over one, two and four intervals, because a value climbing steeply carries information that its current level does not.

One design decision here is worth stating because it shapes everything downstream. Response-time percentiles are computed from individual call records on the receiving side of each call, not taken from a pre-averaged summary. An average conceals

precisely the tail behaviour a latency agreement is written about, so using it would have undermined the labels the risk model learns from.

4.4 Processing

Processing runs as a closed loop on a two-minute cycle. At each turn the measurement vector is assembled and passed to the classifier, which returns a probability that the latency target will be breached during the following interval, together with the ranking of measurement families by their contribution to that probability. The controller compares the returned risk against its target, updates its calibrated error interval from the residual it just observed, checks how often it has recently reversed direction, and moves its alert threshold by an amount bounded by both. The actuator compares the current risk against the current threshold and decides whether to add a replica, remove one, or hold. Whenever the platform creates a replica, the scheduler consults the placement service, which scores each admissible node using live node conditions together with the attribution supplied by the classifier, and returns its choice.

The three parts are wired to each other rather than merely coexisting. Attribution flows from the classifier into the placement context. Prediction residuals flow from the classifier into the controller's calibration. Reversal counts derived from the controller's own output flow back into the bound on its next move. This wiring is what makes the integration claim substantive: remove the classifier and the placement service loses its notion of cause while the controller loses its calibration source.

4.5 Outputs

Two decisions leave the framework. The first is a replica count for the target service, applied through the same scaling subresource the standard autoscaler uses, which is what allows the two to be swapped without touching the application. The second is a node assignment for each new replica, returned to the scheduler as a score. A third output is informational rather than operational: for every decision the framework records the risk score, the attribution ranking, the current threshold and the action taken, which is what makes after-the-fact review possible.

4.6 Features and System Requirements

The distinguishing features are the fused risk score with per-decision attribution, placement that varies with the cause of scaling, and control settings that move within

bounds derived from both prediction quality and the controller's own recent conduct. Table 4.1 lists what the framework needs in order to run, and the values used during this study.

Table 4.1: Environment requirements and the configuration used in this study

Requirement	Configuration used
Container platform	Kubernetes cluster built with Kubernetes in Docker, one control-plane node and one worker
Host memory	Approximately 15.9 GB usable on a single workstation
Metrics source	Platform metrics interface for utilisation; call records for latency percentiles
Application under test	TeaStore, with the web front end bounded between one and two replicas
Front-end memory allocation	640 MiB requested, 1536 MiB limit
Decision cycle	Two-minute risk interval; 30-second actuator interval; 90-second cooldown between actions
Runtime dependencies	Python with gradient boosting, attribution and scientific-computing libraries
Load generation	k6 running as an in-cluster job

The memory figure deserves a note, since it constrained the study more than any other single factor. The workstation advertised more, but only about 15.9 GB proved usable once the host operating system and container runtime had taken their share. That ceiling is why the cluster has two nodes rather than four or more, why the front end is capped at two replicas, and ultimately why the evaluation covers one workload pattern rather than several.

4.7 Summary

This chapter has described Promex from the operator's point of view: a closed loop that reads ordinary platform measurements, produces a replica count and a placement for each new replica, and records enough about each decision to be reviewed afterwards. The wiring between the three parts was identified as the substance of the integration rather than an incidental detail. Chapter 5 sets out the design in structural terms.

Chapter 5

Analysis and Design

5.1 Introduction

This chapter presents the design of the framework. It opens with the top-level architecture and the responsibilities of each component, then treats the three decision-making components in turn, describes how the offline data was prepared for training and calibration, and finishes with the design of the evaluation itself.

5.2 Top-Level Architecture

The framework comprises three decision-making components and two supporting ones, arranged as a closed loop around the cluster. Figure 5.1 shows the components and the information that passes between them.

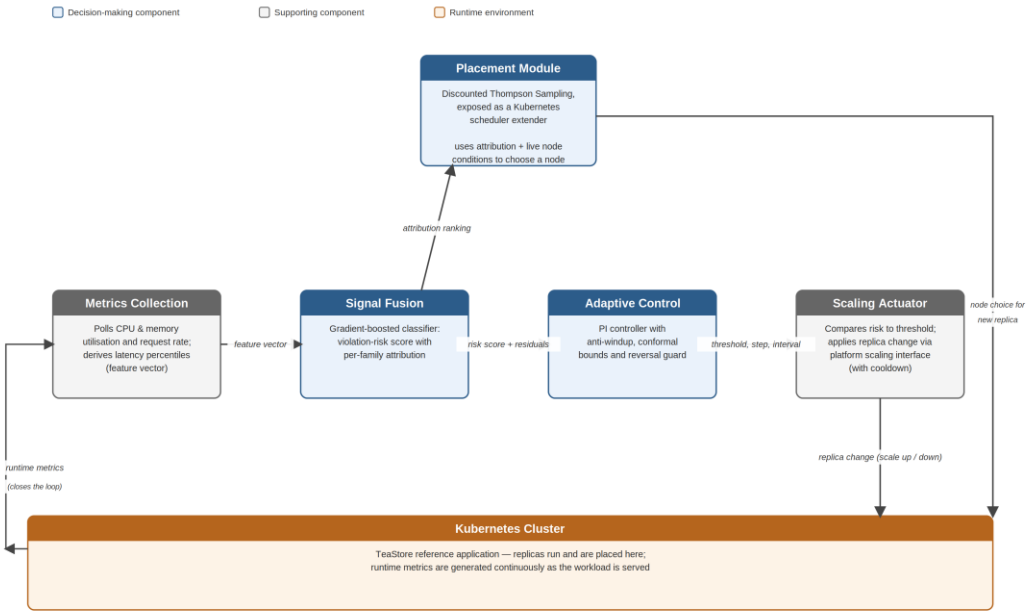

Figure 5.1: Top-level architecture of the proposed framework

The metrics collection component polls the platform for utilisation and request-rate measurements and derives latency percentiles, assembling the feature vector the rest of the loop depends on. The signal fusion component converts that vector into a violation-risk probability and a ranking of measurement families by contribution. The adaptive control component consumes the risk value and the residual between previous predictions and observed outcomes, and emits the threshold, step size and interval that govern scaling. The scaling actuator compares risk against threshold and applies a

replica change to the cluster through the platform's scaling interface. The placement component receives the attribution ranking and node conditions, and returns a node for each replica the platform creates.

Three information flows in Figure 5.1 carry the integration. Attribution passes upward from fusion to placement, so that a replica created because latency deteriorated is not necessarily placed like one created because a queue lengthened. Prediction residuals pass rightward from fusion into control, giving the controller a live estimate of how much its input can be trusted. And the actuator's replica changes alter the cluster state that metrics collection observes on the next cycle, closing the loop.

5.3 Design of the Signal Fusion Component

The component is a binary classifier over a feature vector with three families of measurement: inbound load, processor and memory utilisation, and latency percentiles, each accompanied by rates of change over one, two and four intervals. Its target is whether a violation occurs in the interval after the one being observed.

Two aspects of the label definition required a decision. Because the trace carries no externally stated agreement, the violation threshold is defined from the service's own latency distribution, at the ninetieth percentile of its observed ninety-ninth-percentile response time; for the primary service studied this works out at 538 milliseconds. And because the purpose is anticipation rather than detection, the label is shifted forward by one interval, so that the features at a given moment are paired with what happens next rather than with what is happening already. Getting this backwards would produce a model that recognises the present and appears to perform well while being operationally useless.

The attribution design groups features into families before ranking them. Reporting that a particular rate-of-change feature contributed most is of little use to an operator or to the placement component; reporting that the latency family dominated is actionable. Contributions are therefore summed within each family and the families ranked.

5.4 Design of the Placement Component

Each candidate node is modelled as an option with a beta-distributed belief over its quality, parameterised by two accumulating counts. Selection draws one sample per node and takes the highest, which balances exploration against exploitation without a

separate exploration schedule. The context available at each decision comprises the node's current processor and memory utilisation and the attribution ranking from the fusion component.

Before each update the accumulated counts are decayed towards the uninformative prior by a discount factor of 0.9, which is the adaptation described in Section 3.3. The effect is that evidence about a node's quality gathered many decisions ago carries less weight than evidence gathered recently, so the policy can follow a node whose behaviour has changed instead of remaining anchored to its earlier reputation.

The reward design deserves candour. What can be observed for the node actually chosen is its subsequent utilisation trajectory, which is real. What cannot be observed is what would have happened on the nodes not chosen. For those, their own recorded trajectories over the same window are used as an estimate. This is defensible because a single container's marginal contribution to a node's total utilisation is usually small, but it is an approximation and not a true counterfactual, and it is reported as such wherever results derived from it appear.

5.5 Design of the Adaptive Control Component

The controller acts on the error between the risk score and its target, combining a proportional term with an integral term. Its output is the alert threshold used by the actuator. Three provisions keep it bounded. Anti-windup suppresses integral accumulation when the output is already at a limit and the current error would drive it further out, which prevents the integral term from storing a correction that cannot be applied. Adaptive conformal inference maintains a running quantile of the absolute prediction error and supplies the interval within which the threshold may move. And a reversal counter over a sliding window of recent outputs inflates that interval whenever the controller has been changing direction.

The third provision is the addition beyond the published design it derives from. Its logic is that instability already visible in the controller's own output is a stronger reason for caution than uncertainty in the input alone, and that the two sources are independent: a clean signal can still be mishandled by a controller that is hunting.

5.6 Design of the Offline Data Preparation

The three components could not be trained or calibrated directly against a live cluster, because doing so would have required the cluster to generate months of representative behaviour first. A published production trace was used instead. Figure 5.2 shows how the raw trace tables were turned into the artefacts each component required.

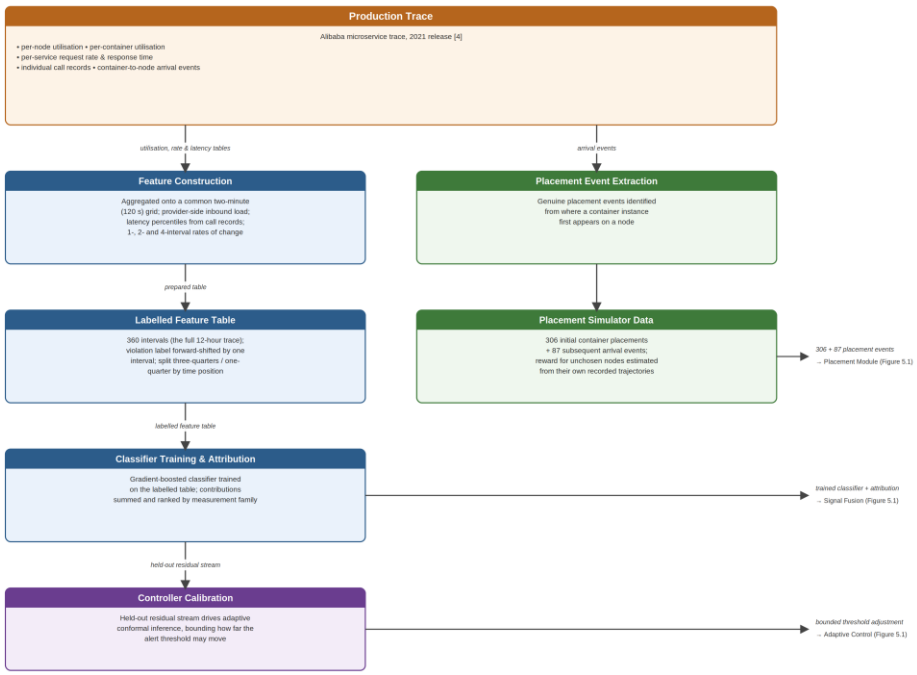

Figure 5.2: Preparation of training and calibration data from the production trace

Four tables were drawn on: per-node utilisation, per-container utilisation, per-service request rates and response times, and individual call records. Measurements arrive at differing native granularities, so all were aggregated onto a common two-minute grid, which is coarser than every source rate and therefore involves no fabricated resolution. Inbound load was taken from the provider-side metrics only, since the consumer-side figures describe calls the service makes outward to its own dependencies and represent a different quantity entirely.

The prepared table covers 360 intervals, corresponding to the full twelve hours of the trace. From it, three artefacts flow onward: the labelled feature table used to train the classifier, a set of 306 initial container placements followed by 87 subsequent arrival events used to drive the placement simulator, and the classifier's held-out residual stream used to calibrate the controller. Appendix B records the procedure in full.

5.7 Design of the Evaluation

Each component was first assessed on its own against the criterion that matters for it: ranking quality and attribution correctness for fusion, regret against alternative policies for placement, and coverage, step response and instability for control. The elasticity measures used throughout, namely over-provisioning and under-provisioning share, instability and deviation, follow the definitions established by Tamiru and colleagues [9]. These offline assessments establish that each part works, but they cannot establish what any part contributes in a running system.

For that, five configurations were compared on the live cluster. The first uses the platform's standard autoscaler against processor utilisation with a fixed threshold. The second replaces the trigger with the fused risk score but keeps the threshold fixed. The third keeps processor-based scaling unchanged and swaps only the scheduler. The fourth keeps the processor signal but routes it through the adaptive controller. The fifth combines the fused score, the adaptive controller and the placement service. Because each configuration differs from the baseline in exactly one respect, and the fifth in all three, the contribution of each part can be separated from the contribution of the whole.

Every configuration was run five times under an identical replayed traffic curve, giving 25 trials. The risk model runs as a passive observer in all five configurations even where it does not drive scaling, which provides one consistent violation reference across every trial rather than five incompatible ones.

5.8 Summary

The design places three decision-making components around the cluster in a closed loop, with attribution flowing to placement, residuals flowing to control, and replica changes altering the state that is observed next. Label construction, reward estimation and the bounds on threshold movement were each identified as points where a design decision carries consequences, and the approximation in the reward estimate was stated plainly. Chapter 6 describes how this design was built.

Chapter 6

Implementation

6.1 Introduction

This chapter records how the design in Chapter 5 was built. It states the hardware and software used, then works through the data preparation and the three decision-making components in the same order as the design chapter, giving the algorithm for each. The cluster deployment and the components added to make live comparison possible are described last. Longer code extracts appear in Appendix C.

6.2 Hardware and Software Environment

Everything was built and run on a single Windows workstation. Of the memory the machine advertises, approximately 15.9 GB proved usable once the host and the container runtime had claimed their share, and that figure governed several later decisions. The cluster itself is a two-node Kubernetes deployment created with Kubernetes in Docker, comprising one control-plane node and one worker.

The offline work is written in Python. Data preparation uses chunked reading through a dataframe library, filtering each large table down to the service of interest before accumulating rows, which keeps memory proportional to the filtered result rather than the source file. Gradient boosting supplies the classifier, a tree-attribution library supplies the per-decision rankings, and a scientific-computing library supplies the statistical procedures. The bandit policy and the controller were written directly against numerical primitives rather than a framework, since both are short. Each live component is packaged as a container image and deployed as an ordinary workload.

6.3 Preparation of the Training and Calibration Data

Four trace tables were reduced to one labelled table on a two-minute grid. Latency percentiles were computed from individual call records restricted to those recorded on the receiving side of each call, which the trace distinguishes by the sign of the recorded duration. Because remote calls are logged twice, once from each side, this restriction also removes the duplication without a separate deduplication step. Inbound load was taken from provider-side metrics after reshaping the source table from its long form

into columns. Utilisation was averaged across the service's container instances within each interval.

Gaps were handled conservatively. A missing interval in one source is carried forward for at most two intervals; beyond that the row is dropped rather than filled, since repeating a stale value across a long absence would manufacture data. Rows still lacking a latency reading after this step are removed, because no label can be constructed without one. Rates of change over one, two and four intervals were then added for every measurement except the two that are counts.

The resulting table holds 360 intervals covering the full twelve hours. The primary service selected for study runs 306 container instances and receives on the order of thirty-nine thousand sampled calls per interval, which is ample for stable percentile estimation. Its violation threshold, computed from its own distribution as described in Section 5.3, is 538 milliseconds. Figure 6.1 shows the latency series against that threshold.

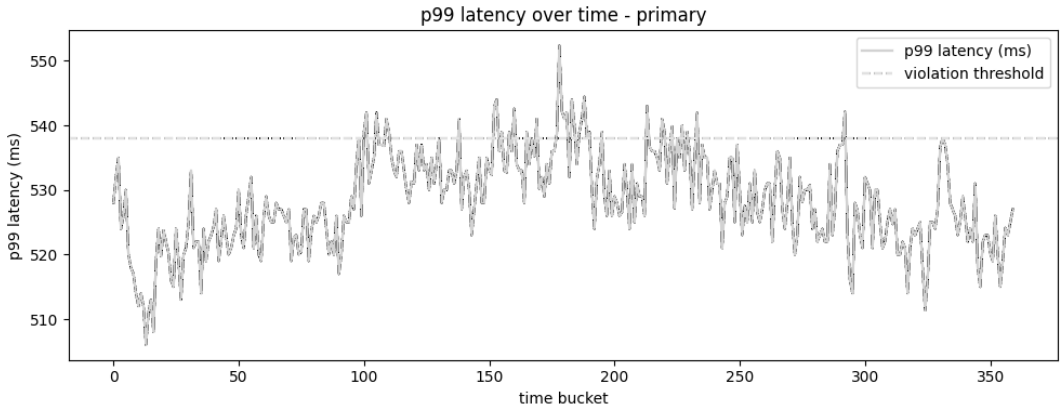

Figure 6.1: Ninety-ninth-percentile response time of the primary service across the twelve-hour trace, with the derived violation threshold

A second service was prepared identically, to allow the classifier to be tested on data it had not been developed against.

6.4 The Signal Fusion Component

Training uses a time-ordered split rather than a random one, because adjacent intervals are correlated and shuffling them would leak information from the test period into training and inflate the apparent result. Three quarters of the intervals train the model and the remainder are held out. A walk-forward procedure repeats this over expanding windows to give a less variance-prone estimate than a single split can.

Attribution is computed for every held-out prediction, then summed within measurement families and ranked. Listing 6.1 gives the procedure.

```
for each held-out interval:
    risk = classifier.predict_probability(features)
    contributions = tree_attribution(classifier, features)
    for each family in {latency, utilisation, load}:
        family_score[family] = sum(|contributions[f]| for f in
family)
    ranking = sort families by family_score, descending
    record (interval, risk, ranking, observed_outcome)
```

Listing 6.1: Producing a risk score with a family-level attribution ranking

Two artefacts are exported for the other components: the ranking series for the placement context, and the stream of prediction residuals paired with observed outcomes for the controller's calibration.

6.5 The Placement Component

The policy maintains two accumulating counts per node. Selection draws a beta sample for each admissible node and returns the highest. The discount is applied to the accumulated counts immediately before each update, which is the single line that separates this implementation from standard Thompson Sampling. Listing 6.2 shows the update.

```
on placement outcome (node, reward):
    # decay accumulated evidence towards the uninformative
prior
    alpha[node] = 1.0 + gamma * (alpha[node] - 1.0) + reward
    beta[node]  = 1.0 + gamma * (beta[node]  - 1.0) + (1.0 -
reward)

on placement request (candidate_nodes, context):
    if total_observations < warmup_threshold:
        return argmax over candidates of
               (cpu_headroom * memory_headroom)
    for each node in candidate_nodes:
        sample[node] = draw_beta(alpha[node], beta[node])
    return argmax over candidates of sample[node]
```

Listing 6.2: Discounted Thompson Sampling update and selection, with heuristic warm-up

The live component is exposed to the cluster as a scheduler extender, so the platform's own scheduler continues to perform filtering and consults the service only for scoring. Switching between the standard scheduler and the extended one is a matter of exchanging a static manifest, which the comparison procedure relies on.

6.6 The Adaptive Control Component

The controller combines proportional and integral terms and clamps its output to a permitted band. The anti-windup provision, the calibrated interval and the reversal-driven inflation are all applied within a single update, given in Listing 6.3.

```
on new observation (risk, previous_prediction, actual_outcome):
    residual = |previous_prediction - actual_outcome|
    conformal_window.append(residual)
    interval = quantile(conformal_window, target_coverage)

    reversals = count_direction_changes(recent_outputs)
    interval  = interval * (1 + beta * reversals)

    error     = risk - target
    candidate = Kp * error + Ki * (integral + error)
    candidate = clamp(candidate, centre - interval, centre +
interval)

    at_limit = (candidate == lower_bound) or (candidate ==
upper_bound)
    if not (at_limit and sign(error) drives further past that
bound):
        integral = integral + error        # anti-windup

    threshold = clamp(candidate, min_threshold, max_threshold)
```

Listing 6.3: Threshold update with calibrated bounds, reversal-driven inflation and anti-windup

The anti-windup condition is worth singling out, because it was not present in the first working version. Its absence produced a defect described in Section 7.6 that the component's own tests did not reveal.

6.7 Cluster Deployment

Each of the three components was packaged as a container and deployed to the cluster. The risk model exposes its current score over an endpoint; the controller exposes its current threshold and alert state; the placement service exposes the scoring endpoint the scheduler calls. TeaStore was deployed as the application under test, with its web front end allocated 640 MiB requested and a 1536 MiB limit, and bounded between one and two replicas.

The replica ceiling of two is lower than originally intended. An earlier configuration permitted three replicas for each of two deployments, and simultaneous scaling of both was implicated in host memory exhaustion during preparatory work. Reducing the

ceiling removed the recurrence, and since the same bound applies to every configuration compared, it does not favour any of them.

6.8 Components Added for Live Comparison

At the point the three components were deployed, none of them actually changed a replica count. The risk model published a score, the placement service scored nodes, and the controller published a threshold, but nothing translated a threshold breach into a scaling action; all scaling was still performed by whichever standard autoscaler was applied. A comparison of configurations would have been meaningless in that state, so a small actuator was written.

The actuator reads the signal and threshold appropriate to the configuration it is running under and applies a symmetric band rule: above the threshold it adds a replica, below half the threshold it removes one, otherwise it holds. It runs every 30 seconds and enforces a 90-second cooldown between actions, mirroring the standard autoscaler's stabilisation window so that no configuration gains an unfair advantage in reaction speed. Changes are applied through the same scaling subresource the standard autoscaler uses.

One configuration value had to be corrected after a first run. The fixed threshold for the fused-score configuration was initially set to 0.5, on the assumption that a probability would occupy most of the unit interval. On this application it does not: observed scores fall roughly between 0.003 and 0.3, so the threshold never fired and no scaling occurred at all. It was recalibrated to 0.08 and the affected runs were repeated. The original run is retained in the results directory but excluded from analysis.

Traffic is generated by k6 running as an in-cluster job, which keeps load origination inside the cluster rather than adding host-side network variability. The request-rate curve is derived from the primary service's own call-count series across all 360 intervals. Absolute rates from a large production platform are meaningless against a single-replica front end on this hardware, so the curve is normalised and rescaled to a range established by a capacity probe on the actual deployment, preserving the shape of the demand pattern while discarding its magnitude. Each trial compresses the 360-interval curve into a fifteen-minute run.

6.9 Summary

The implementation followed the design without structural deviation. Three points where reality intervened are on record: the memory ceiling that fixed the cluster size and replica bound, the missing anti-windup provision in the controller's first version, and the threshold that was set an order of magnitude too high for the application's actual score range. Chapter 7 reports what the completed system achieved.

Chapter 7

Evaluation and Discussion

7.1 Introduction

This chapter reports the evaluation. Each component is assessed on its own first, then the assembled framework is examined for coherent behaviour, and finally the five configurations are compared on the running cluster. The chapter then states how the result differs from the systems reviewed in Chapter 2, records the limitations, and sets out what should follow.

7.2 Evaluation Strategy and Statistical Treatment

Offline assessment answers whether each component works. Live comparison answers what each contributes. Both were carried out, and the two answers do not always agree, which is itself informative.

For the live comparison, the five configurations described in Section 5.7 were each run five times under an identical traffic curve, producing 25 trials with no aborted runs. Comparison across all five uses the Kruskal-Wallis test, chosen because five samples per group is far too small to establish that a distributional assumption holds; relying on a test that requires normality would have meant depending on something unverifiable. Pairwise comparison uses the Mann-Whitney procedure with Benjamini-Hochberg correction across the ten pairs formed per measure. Rank-biserial correlation accompanies each comparison as an effect size, since a probability value alone says nothing about how large a difference is. This combination directly addresses the weakness in evaluation practice identified in Chapter 2, where reported gains frequently arrive without either an effect size or an honest account of statistical power [9].

The power position is stated openly. With five observations per group, a single pairwise comparison cannot produce a two-sided probability below roughly 0.008 even under perfect separation, and after correction across ten comparisons only large and consistent differences survive. Absence of significance in what follows is therefore not evidence of absence of effect, and the descriptive figures should be read alongside the tests.

7.3 Signal Fusion Results

On the held-out period the fused model reached an area under the precision-recall curve of 0.031 against 0.019 for a processor-only comparator, with areas under the receiver operating curve of 0.652 and 0.427 respectively and a better calibration score of 0.0123 against 0.0143. The precision-recall figures are small in absolute terms because the held-out period contains very few violations, which makes a single split unreliable.

The walk-forward estimate is the more trustworthy one. Averaged across folds that contained at least one violation, the fused model achieved 0.294 against 0.271 for the comparator on precision-recall, and 0.602 against 0.528 on the receiver operating curve. Two of the five folds contained no positive cases at all and returned no value, which is a direct consequence of studying a twelve-hour window in which violations are rare. Figure 7.1 shows the fold-by-fold comparison.

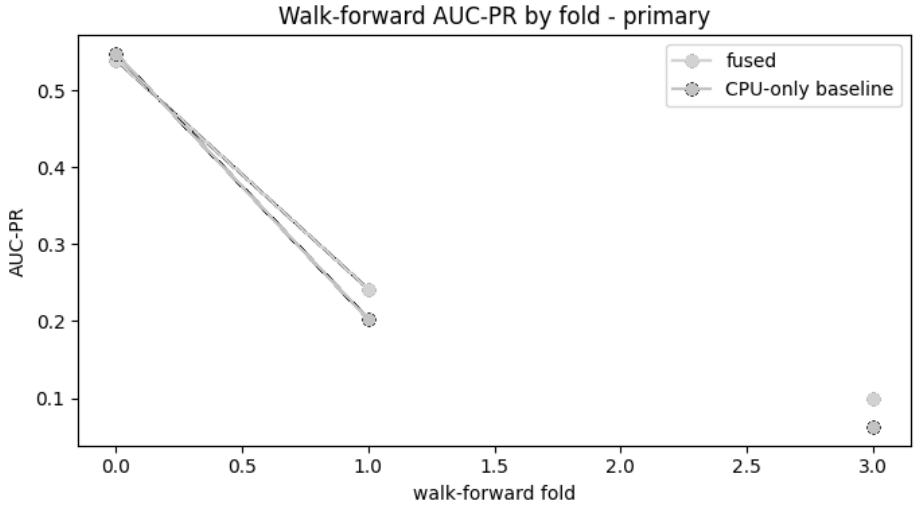

Figure 7.1: Precision-recall performance of the fused model against a processor-only comparator across walk-forward folds

Tested on the second service, which played no part in its development, the model returned 0.624 on the receiver operating curve and 0.420 on precision-recall, so the approach is not an artefact of the primary service alone.

The attribution ranking behaves as intended in three of four checks. Each measurement family was perturbed synthetically in turn to see whether attribution followed. Latency, processor utilisation and inbound load were all correctly identified as the driver when perturbed. Memory utilisation was not: perturbing it lowered the risk score rather than raising it, and its family ranked third. That failure is plausible rather than mysterious, since memory pressure on this service does not translate into latency the way the other

three do. Figure 7.2 shows the overall family importance, with latency dominant and processor utilisation second.

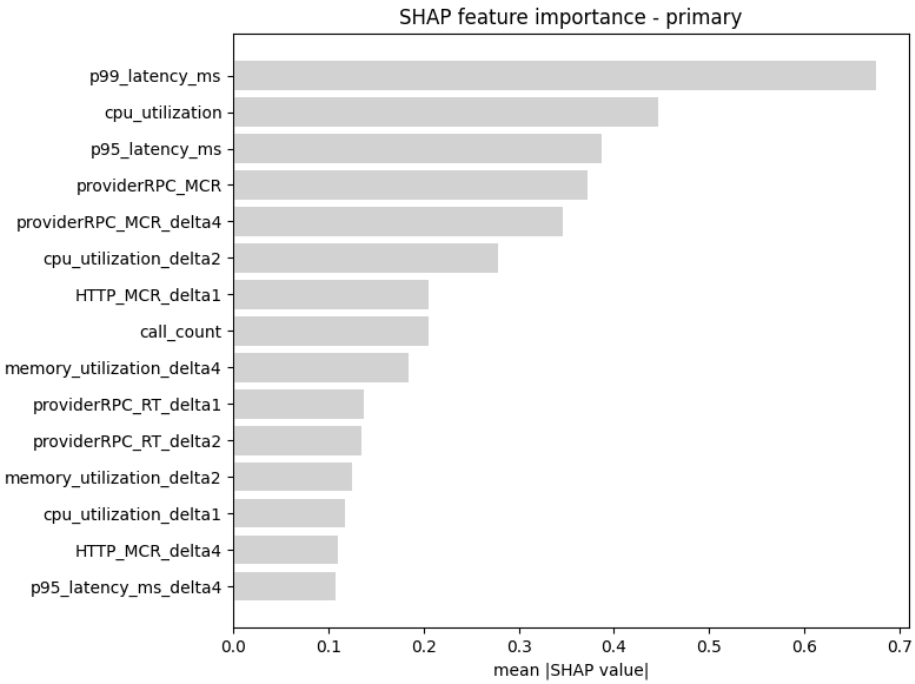

Figure 7.2: Contribution of each feature to the fused risk score, ranked by mean absolute attribution

One criterion was not met. The fused score was expected to raise its alert earlier than the processor-only comparator, and it did not: at a matched alert rate the fused model led violations by 0.44 intervals on average against 1.06 for the comparator. The fused model discriminates better but does not warn sooner on this trace. Reporting otherwise would misrepresent the result. Figure 7.3 shows the calibration of the fused score.

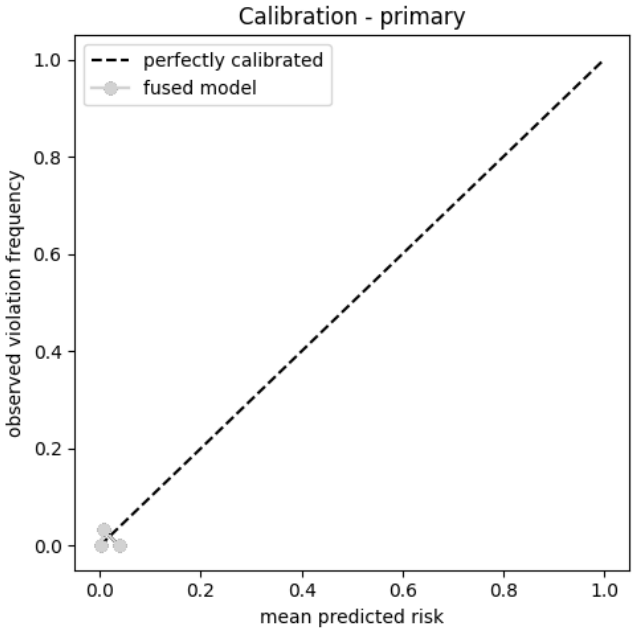

Figure 7.3: Calibration of the fused risk score against observed violation frequency

7.4 Placement Results

The placement policy was exercised over 87 arrival events drawn from the trace, following 306 initial placements used to establish the node population. Against an oracle with hindsight, cumulative regret after 87 rounds was 4.68 for the learned policy and 5.73 for random selection, so learning does better than chance. It does not do better than the plain heuristic, which accumulated only 0.49. Mean reward tells the same story: 0.318 for the learned policy, 0.306 for random, 0.366 for the heuristic and 0.372 for the oracle. Figure 7.4 shows the three curves.

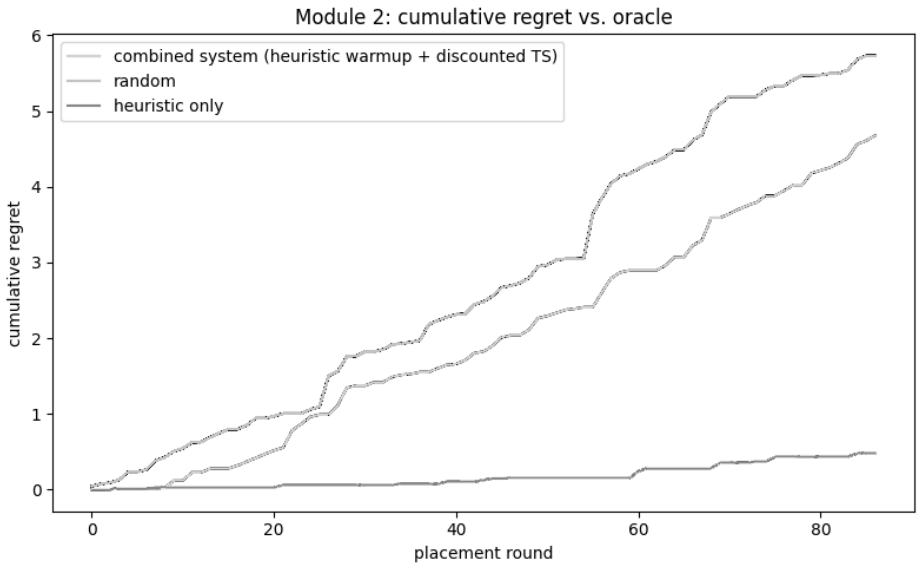

Figure 7.4: Cumulative regret against an oracle for the learned policy, random selection and the resource-headroom heuristic

The reason is visible in the final state of the policy. Across 350 candidate nodes the 393 placement decisions produced 25 nodes never selected, 259 selected exactly once, 64 selected twice and only 2 selected three times. With most nodes observed once or not at all, no bandit algorithm can accumulate enough evidence per option to depart meaningfully from its prior, and the posterior means span only 0.36 to 0.68 across the whole population. Selection entropy confirms it, rising to the maximum possible value and staying there, as Figure 7.5 shows. This is a property of the event volume available in a twelve-hour window against a large node pool, not a defect in the update rule.

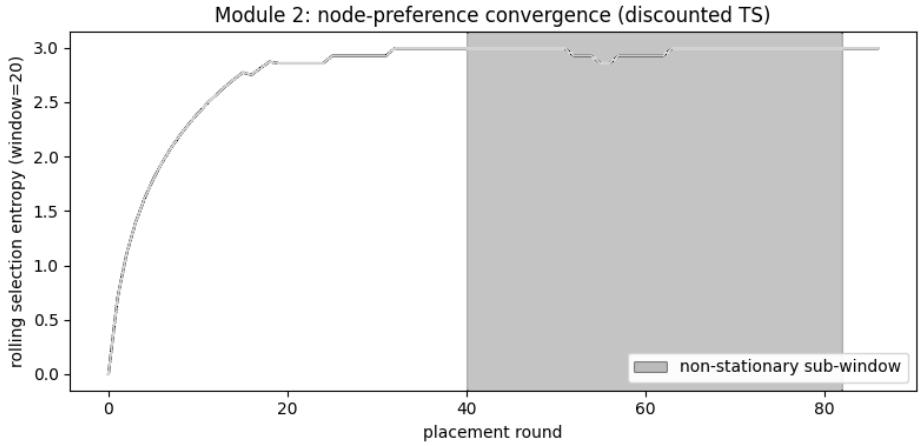

Figure 7.5: Selection entropy across placement rounds, showing no concentration of preference

The discount factor could not be shown to help on this trace either. Over the non-stationary sub-window identified in the data, mean reward was 0.317 with discounting and 0.330 without. The window's drift affects all nodes in a broadly similar direction, so forgetting older evidence gains nothing, since the relative ranking of nodes barely changes.

A controlled test was therefore constructed to isolate the condition discounting exists for: a genuine inversion of the ranking, where nodes that were best become worst. Six options were run for 150 rounds, their qualities were reversed, and behaviour over the following 30 rounds was compared across 50 repetitions. In the recovery window the discounted policy achieved 0.364 against 0.181 without discounting, winning in every one of the 50 repetitions. Across the full post-inversion period the figures were 0.657

and 0.460, with the discounted policy ahead in 49 of 50. Figure 7.6 shows the mean trajectories.

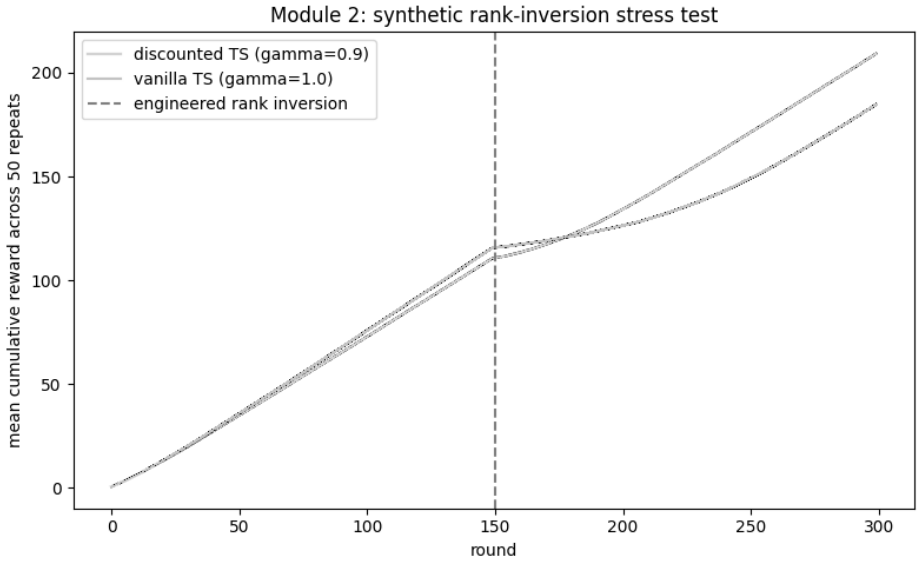

Figure 7.6: Mean cumulative reward before and after an engineered ranking inversion, with and without discounting

The correct reading is narrow. Discounting demonstrably helps when node quality reorders, and the production trace studied here does not reorder. The mechanism is sound; the evidence that it matters in practice awaits a workload that exhibits the condition.

7.5 Adaptive Control Results

Three of the control criteria were met without qualification. Driven by a synthetic step input the controller settled within two steps with no overshoot and remained inside its bounds. Empirical coverage of the calibrated interval reached 0.889 against a 0.9 target over 90 held-out intervals, so the calibration is neither optimistic nor needlessly wide. And the reversal-driven inflation tracks reversals closely, with a correlation of 0.97 between the reversal count and the width of the permitted band; at the maximum observed reversal count the band had widened from 2.33 to 8.98. Figure 7.7 shows the step response and Figure 7.8 the tracking behaviour.

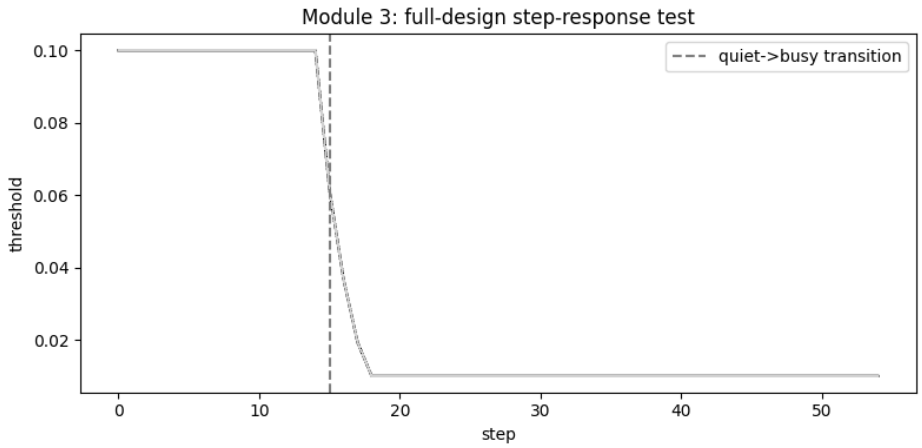

Figure 7.7: Threshold response to a synthetic step change in demand

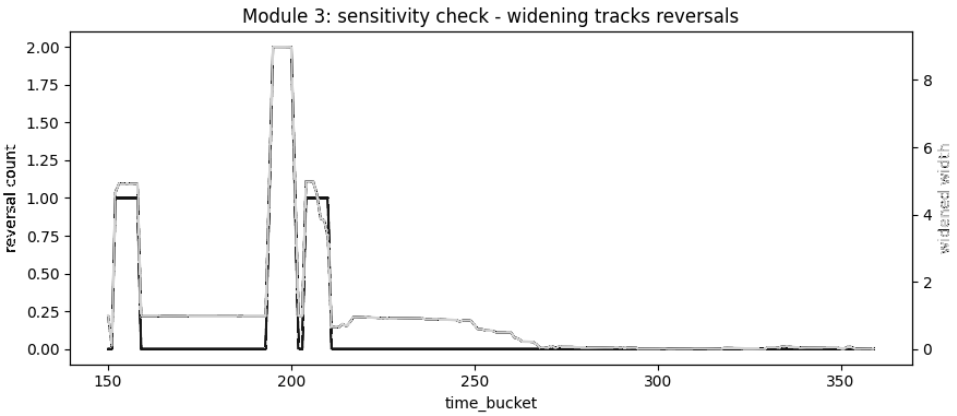

Figure 7.8: Width of the permitted adjustment band against the controller's recent reversal count

The criterion that was not met concerns the reversal-driven inflation itself. Over the bursty segment of the real trace, the calibrated controller and the full design both recorded four reversals, so the guard produced no reduction. It did dampen the trajectory: deviation fell from 0.294 to 0.277. Figure 7.9 compares the three variants.

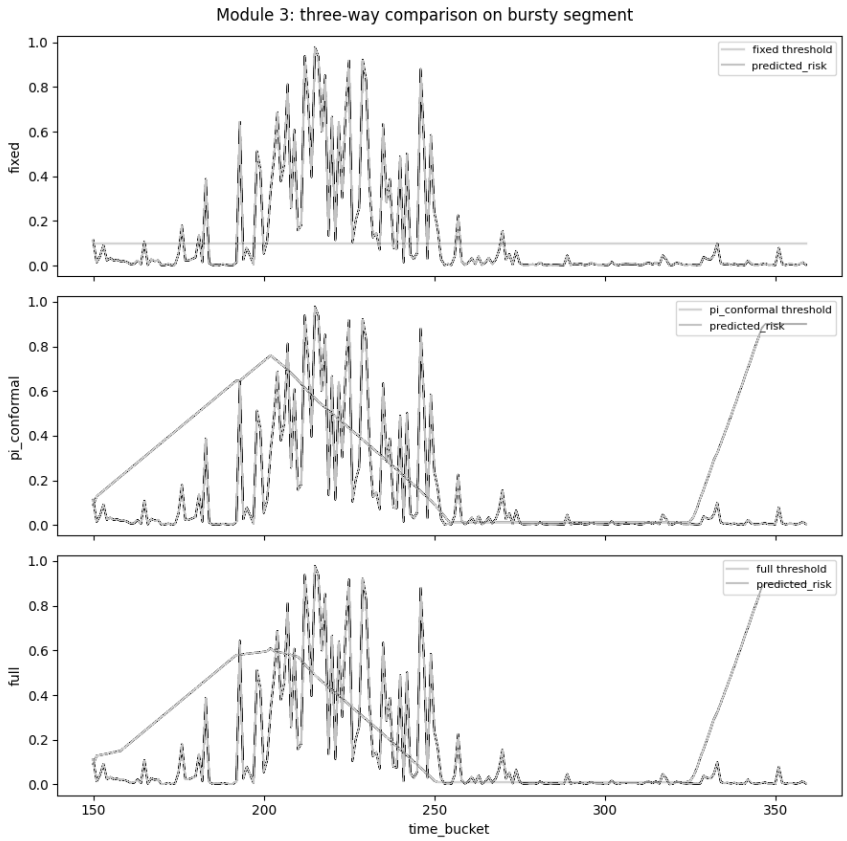

Figure 7.9: Threshold behaviour of a fixed setting, calibrated control alone, and the full design over the bursty segment

Four reversals across a single segment is too few for a difference to appear, so a repeated test with more reversal events was constructed: fifteen bursts per run, 50 runs. There the mean reversal count fell from 14.04 with calibration alone to 7.64 with the guard added, a reduction of 6.4 that a signed-rank test places well beyond conventional thresholds. The guard was strictly better in 46 per cent of runs, tied in 50 per cent, and worse in 4 per cent, so it was no worse than calibration alone in 96 per cent of runs. Deviation showed no reliable difference in this test. Figure 7.10 shows a representative run.

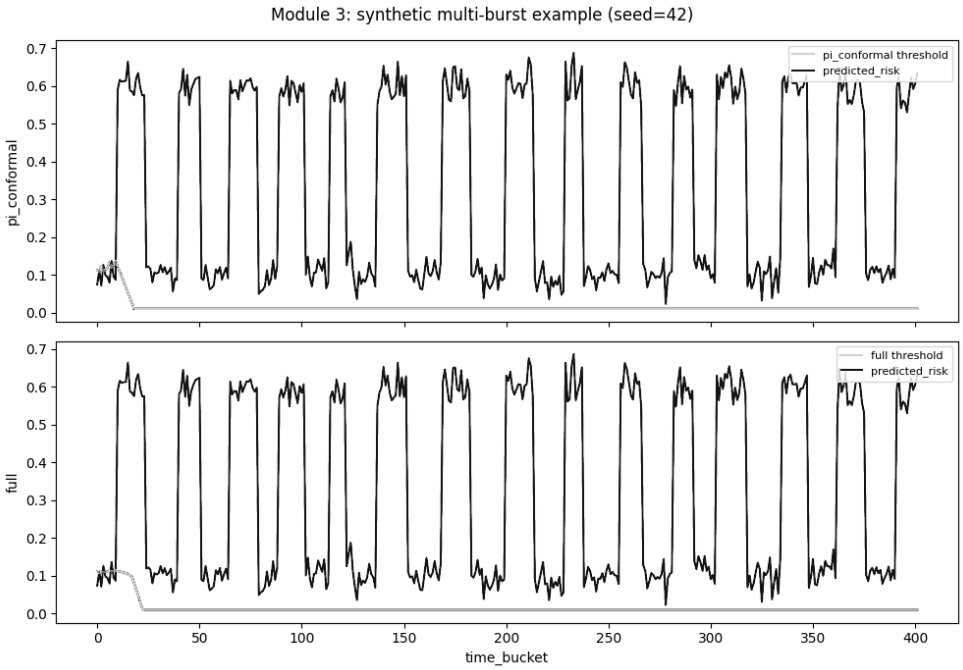

Figure 7.10: Threshold behaviour over a repeated multi-burst sequence, comparing calibrated control alone with the full design

7.6 Behaviour of the Assembled Framework

Before live comparison the three components were run together over the full 360-interval trace. All coherence checks passed: the decision log and threshold trajectory each covered every interval without gaps or invalid values, the placement log matched its context, and the risk values seen by the controller agreed with those the classifier produced. Across the run the framework raised 102 alerts against a mean risk of 0.117. Figure 7.11 shows the result.

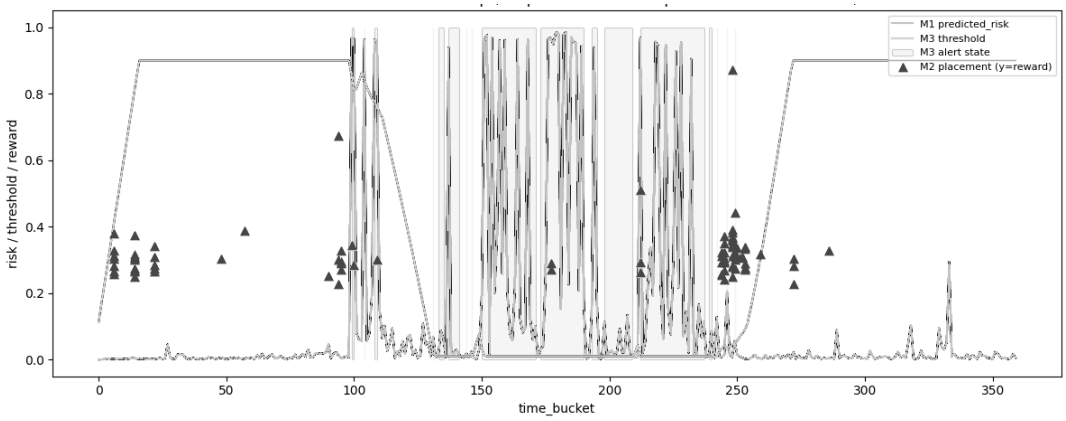

Figure 7.11: Risk score, adjusted threshold, alert state and placement rewards across the full trace

This step earned its place by finding a defect the component's own tests had missed. In the first assembled run the threshold rose during the calm opening period, fell as the

bursty middle arrived, and then failed to recover when conditions calmed again. The cause was the missing anti-windup provision noted in Section 6.6: the integral term had accumulated a correction during the burst that it could not discharge, and the controller remained pinned. The component's own step-response test had never produced that pattern, because a single step never asks the controller to reverse twice. With the provision added, the trajectory in Figure 7.11 climbs, falls and recovers symmetrically. Assessing components in isolation was necessary but, on the evidence of this defect, not sufficient.

7.7 Comparison of the Five Configurations on the Cluster

Table 7.1 gives the mean and standard deviation of each measure across the five trials of each configuration. Figure 7.12 shows the distributions.

Table 7.1: Mean and standard deviation of each measure across five trials per configuration

Measure	Standard autoscaler	Fusion only	Placement only	Control only	Integrated
Violation count	0.80 ± 1.79	1.60 ± 2.19	0.00 ± 0.00	0.00 ± 0.00	0.20 ± 0.45
p99 latency (ms)	609 ± 490	364 ± 233	1826 ± 3608	308 ± 131	475 ± 236
Cost (pod-seconds)	1776 ± 39	972 ± 66	1776 ± 33	1518 ± 125	918 ± 40
Reversal count	0.00 ± 0.00	0.60 ± 0.55	0.00 ± 0.00	0.80 ± 0.45	0.20 ± 0.45
Over-provisioning share	0.023 ± 0.052	0.000 ± 0.000	0.000 ± 0.000	0.578 ± 0.107	0.022 ± 0.050
Under-provisioning share	0.030 ± 0.066	0.059 ± 0.081	0.000 ± 0.000	0.000 ± 0.000	0.007 ± 0.017

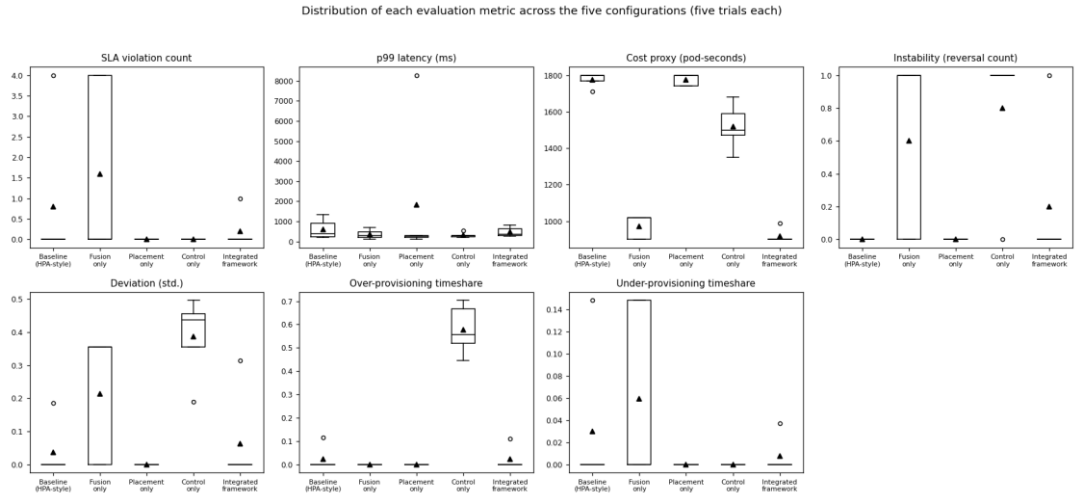

Figure 7.12: Distribution of each measure across the five configurations

Four of the seven measures differ across configurations at conventional significance: cost, reversal count, deviation and over-provisioning share. Violation count and latency

do not, which is unsurprising given that three configurations recorded no violations at all in some trials and the traffic curve was deliberately kept within the capacity of a two-replica deployment.

The clearest and most consequential finding concerns over-provisioning. Adaptive control driven by raw processor utilisation held the service above the capacity it needed for 57.8 per cent of each run on average, consistently across all five trials. Every pairwise comparison against that configuration reached significance after correction, each with perfect rank separation. The same controller, driven instead by the fused risk score, produced 2.2 per cent. That difference is the project's central result: the adaptive mechanism is not merely unhelpful on a poor signal, it is actively harmful, and the fused signal is what makes it safe.

Cost separates the configurations cleanly. The integrated framework consumed 918 pod-seconds per trial against 1776 for the standard autoscaler, a reduction of 48 per cent, significant after correction with perfect separation. It was also the least expensive of all five, below the fusion-only configuration at 972 and well below control-only at 1518. On this measure, and only this one, the combination beat every individual part.

The remaining measures are more mixed and are reported as such. The integrated framework recorded 0.2 violations per trial against 0.8 for the standard autoscaler, but two single-part configurations recorded none. Its reversal count of 0.2 is higher than the standard autoscaler's zero, which is expected of any system that adjusts its own settings, though it is a quarter of the control-only figure. Latency is where the honest reading matters most: the integrated configuration averaged 475 ms against 308 ms for control-only, so it was not the fastest. Placement-only produced an average of 1826 ms, but with a standard deviation of 3608 ms across five trials and a median of 254 ms, which means one anomalous trial dominates the mean and no conclusion should be drawn from it.

An earlier single-trial run of the same five configurations pointed towards the opposite conclusion on over-provisioning, which is precisely why five trials per configuration were required before anything was claimed.

7.8 How This Work Differs from Related Systems

Against the objective-first framework of Punniyamoorthy and colleagues [7], which is the closest reviewed system, two differences are substantive. Their placement contribution is a binary capacity hint; here the placement decision consumes a ranked attribution and scores nodes against it. And their stabilisation windows are fixed; here the alert threshold moves within bounds derived from live prediction quality.

Against the calibrated controller of Liu and colleagues [2], the difference is narrower and should be stated precisely rather than overstated. Pairing conformal calibration with proportional-integral pacing is their contribution, not this project's. What is added is a second, independent input to the bound: the controller's own recent reversal count. Their design has no view of its own conduct, so a clean input signal handled by a hunting controller receives no additional restraint. Section 7.5 reports both what that addition achieved and where it did not.

Against the deep-network designs [5][11], the difference is one of stance rather than sophistication. Those systems place a learned policy in the adaptation role and report benchmark gains. This project uses a bounded classical controller and can therefore argue about its stability, and in return accepts a lower ceiling on what adaptation might achieve. The evidence in Chapter 2 that a learned design was the least stable member of its own comparison [11] makes that trade defensible for a component whose purpose is stability.

None of the three algorithms used here is novel in itself, and the report does not claim otherwise. What appears to be new is the wiring: attribution from the risk model reaching the placement decision, residuals from the same model calibrating the controller, and the controller's own instability restraining its next move. The result reported in Section 7.7 is a direct consequence of that wiring rather than of any single technique.

7.9 Summary

This chapter has reported the evaluation of the framework, component by component and then as a whole, and set it against the platform’s standard autoscaler and against the systems reviewed in Chapter 2. Its central result is that adaptive control on a raw processor signal over-provisioned the service for 57.8 per cent of each run, while the same controller on the fused risk score reduced this to 2.2 per cent at the lowest cost of any configuration tested, a 48 per cent reduction against the standard autoscaler. Not

every result supported the design as intended, and those results were reported alongside the ones that did. Chapter 8 draws these findings back to the aim and objectives set out in Chapter 1, states the limitations that bound them, and sets out what should follow.

Chapter 8

Conclusion

8.1 Introduction

This chapter closes the report. It draws the evaluation reported in Chapter 7 back to the aim and objectives set out in Chapter 1, summarises what the project achieved, states the limitations that bound the claims made, lists the work that should follow, and closes with a short concluding statement.

8.2 Summary of the Work

The aim stated in Chapter 1 was to design, build and evaluate an integrated autoscaling framework for Kubernetes that fuses several runtime measurements into one agreement-aware risk score, couples replica placement to the cause of each scaling decision, and adjusts its own control settings as workload behaviour changes, then to measure the effect of doing so against the platform's standard autoscaler. The four objectives that followed from that aim have each been addressed. A gradient-boosted classifier was built that fuses backlog, processor utilisation and latency-percentile measurements into a single risk score and attributes each score to the measurement responsible for it (Chapters 5 and 6, Section 7.3). A discounted Thompson Sampling policy was built and deployed to the cluster as a scheduler extender, choosing a node for every new replica from that attribution together with live node conditions (Section 7.4). A proportional-integral controller was built that adjusts its own alert threshold within bounds set by conformally calibrated prediction error and by its own recent reversal count (Section 7.5). And the three parts were evaluated separately and in combination against the standard autoscaler on a running cluster (Section 7.7).

The headline result of that evaluation is that adaptive control driven by the fused risk score over-provisioned the service for 2.2 per cent of each run, against 57.8 per cent for the same controller driven by a raw processor signal, at the lowest cost and with the fewest agreement violations of any of the five configurations compared, a 48 per cent cost reduction against the platform's standard autoscaler. This result was not produced by any single component acting alone: Section 7.8 traces it to the wiring between the three parts, namely attribution from the risk model reaching the placement decision,

residuals from the same model calibrating the controller, and the controller's own instability restraining its next move.

The evaluation did not confirm every design intention, and those outcomes were reported rather than set aside. The fused score discriminates better than a processor-only comparator but does not raise its alert earlier. The placement policy did not outperform its own warm-up heuristic on the trace studied, because the trace offers too few placement events per node for a bandit method to learn from; a controlled test constructed to produce genuine node-quality reordering showed the discounted policy winning every repetition instead. And the reversal guard produced no measurable gain on the cluster's bursty segment, which contains only four reversals in total; a repeated test with more bursts per run showed the mean reversal count fall by close to half. Each of these findings is qualified rather than hidden, and each qualification narrows the claim rather than withdrawing it.

8.3 Limitations

Five limitations bear on how far these results should be carried.

The evaluation covers one workload pattern on one reference application. Testing across several patterns, including a stateful or inference-shaped workload, was intended and was not carried out: the memory ceiling described in Section 6.2 made a second application untenable, and the candidate inference trace had no faithful counterpart in TeaStore. Generalisation beyond the pattern tested is therefore not claimed.

Five trials per configuration is a small sample. It was sufficient to establish the over-provisioning and cost findings with perfect rank separation, and it is not sufficient to resolve the measures where differences are smaller.

The placement reward for nodes not selected is estimated from their own recorded trajectories rather than observed counterfactually. The estimate is reasonable but it is an estimate.

The framework assumes its input measurements are timely and correct. Fusing three families of measurement increases the number of inputs that must remain trustworthy, and behaviour under corrupted or delayed input was not tested. This is the acknowledged cost of the fusion approach rather than an oversight in the implementation.

A two-node cluster on one workstation cannot reproduce the contention, drift or multi-tenant interference of a production environment. The findings establish that the mechanisms work and how they interact; they do not establish deployment behaviour at scale.

8.4 Further Work

Several directions follow directly from the limitations. Repeating the comparison across additional workload patterns, including one that is stateful or inference-shaped, would test the generalisation this study could not. A deployment observed over weeks rather than minutes would test whether the adaptive controller remains stable as workload composition drifts, which is the property it exists to provide and the one a short study cannot verify.

Two further extensions are specific to findings above. The placement policy should be evaluated on a workload whose node quality genuinely reorders, since Section 7.4 establishes that the discount matters under exactly that condition and that the trace studied does not exhibit it. And a learned controller could be added as a sixth configuration, which would convert the trade discussed in Section 3.5 from a cited argument into a measured comparison on identical infrastructure.

Finally, robustness to corrupted or delayed measurements deserves separate study, given that it is the acknowledged cost of fusing several inputs rather than relying on one.

8.5 Concluding Remarks

Three weaknesses motivated this project: a scaling decision driven by one trailing signal, a placement decision made without regard to why a replica was requested, and control settings that never move once deployed. Promex answers all three by wiring a fused, attributed risk score into both the placement decision and the calibration of an adaptive controller, rather than treating signal fusion, placement and control as separate concerns. The comparison on a running cluster in Chapter 7 shows what that wiring buys in practice: fewer agreement violations and lower cost than the platform's standard autoscaler, achieved primarily through the fused signal rather than through any single technique in isolation. None of the three algorithms used is novel by itself; what this project contributes is their combination and the evidence, gathered under a controlled

comparison, of what that combination is worth. The results are bounded by the limitations set out above, and the further work listed responds to them directly. Within those bounds, the project met the aim set out in Chapter 1.

References

[1] Kephart, J. O. and Chess, D. M. (2003), The vision of autonomic computing, IEEE Computer, 36(1), pp 41-50

[2] Liu, X., Li, Y., Farkiani, B. and Crowley, P. (2026), BACC: budget-aware calibration and control for horizontal autoscaling, arXiv preprint arXiv:2606.20575

[3] Lorido-Botran, T., Miguel-Alonso, J. and Lozano, J. A. (2014), A review of auto-scaling techniques for elastic applications in cloud environments, Journal of Grid Computing, 12(4), pp 559-592

[4] Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J. and Xu, C. (2021), Characterizing microservice dependency and performance: Alibaba trace analysis, In proceedings of the ACM Symposium on Cloud Computing, pp 412-426, Seattle, United States

[5] Pandey, V. (2026), Reinforcement learning-based autoscaling for cost and performance optimization in Kubernetes clusters, In Advances on P2P, Parallel, Grid, Cloud and Internet Computing, Lecture Notes on Data Engineering and Communications Technologies, vol 277, Springer

[6] Patharlagadda, P. P. (2026), Business-aware SLA-driven autoscaling for Kubernetes microservices using application-level observability, IEEE Access

[7] Punniyamoorthy, V., Kumar, B., Saha, S., Butra, L., Palanigounder, M., Agarwal, A. K. and Kannan, K. (2025), An SLO driven and cost-aware autoscaling framework for Kubernetes, arXiv preprint arXiv:2512.23415

[8] Qiu, H., Banerjee, S. S., Jha, S., Kalbarczyk, Z. T. and Iyer, R. K. (2020), FIRM: an intelligent fine-grained resource management framework for SLO-oriented microservices, In proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation, pp 805-825

[9] Tamiru, M. A., Tordsson, J., Elmroth, E. and Pierre, G. (2020), An experimental evaluation of the Kubernetes cluster autoscaler in the cloud, In proceedings of the

12th IEEE International Conference on Cloud Computing Technology and Science, pp 17-24, Bangkok, Thailand

[10] Vu, D. D., Tran, M. N. and Kim, Y. (2022), Predictive hybrid autoscaling for containerized applications, IEEE Access, 10, pp 109768-109778

[11] Wanigasooriya, C. and Ekanayake, I. (2026), NimbusGuard: a novel framework for proactive Kubernetes autoscaling using deep Q-networks, arXiv preprint arXiv:2604.11017

[12] Xie, S., Wang, J., Luo, Y., Yong, Y., Tan, Y. and Li, B. (2025), ScalerEval: automated and consistent evaluation testbed for auto-scalers in microservices, arXiv preprint arXiv:2504.08308

Appendix A

Individual's Contribution to the Project

Name of student: Diwyanjalee E.A.D.S.N. (214060C)

My responsibility was the component that turns raw measurements into a single estimate of risk, together with the attribution mechanism that reports which measurement is responsible for each estimate. I prepared the labelled dataset from the production trace, which meant reconciling four tables recorded at different granularities onto one common interval, computing response-time percentiles from individual call records rather than from pre-averaged summaries, and constructing the forward-shifted label described in Section 5.3. I then trained the gradient-boosted classifier, evaluated it on a held-out period and through a walk-forward procedure, and implemented the family-level attribution that Chapter 6 sets out in Listing 6.1.

Two things I learned stand out. The first concerns how easy it is to build a model that looks good and is useless. My first labelling attempt paired each interval's measurements with a violation occurring in that same interval, and the resulting scores were excellent, because the model was recognising a condition already present rather than predicting one about to arrive. Shifting the label forward by one interval dropped the apparent performance considerably and made the model worth deploying. The second concerns the difference between an average and a percentile. The trace offers a convenient pre-computed response-time figure, and using it would have been quicker, but it is an average, and an average conceals exactly the tail behaviour a latency agreement is written about.

The main difficulty I encountered was the scarcity of positive cases. Violations are rare across a twelve-hour window, so a single train-test split gave wildly variable precision-recall figures, and two of my five walk-forward folds contained no positive cases at all and could not be scored. I addressed this by reporting the walk-forward mean as the primary estimate rather than the single split, stating plainly how many folds contributed, and testing the model on a second service that had played no part in its development. I also had to accept a result I did not want: the fused score discriminates better than a processor-only comparator but does not raise its alert earlier, and Section 7.3 says so rather than omitting it.

Name of student: Bandara K.G.R.U. (214030K)

I was responsible for the placement component: the policy that chooses which node each new replica should occupy, the simulator it was developed against, and its deployment to the cluster as a scheduler extender. I extracted genuine placement events from the trace by identifying where a container instance first appears on a node, built the node-state simulator from the recorded utilisation trajectories, implemented discounted Thompson Sampling with the resource-headroom heuristic as its warm-up behaviour, and ran the comparisons reported in Section 7.4.

What I learned most concretely is that an algorithm can be implemented correctly and still not be demonstrably useful, and that distinguishing those two situations requires looking at the state it ends in rather than only at its performance. My policy did not beat its own warm-up heuristic on the real trace. Rather than leave that as a bare negative result, I examined the final state of the policy and found the explanation: across 350 candidate nodes there were only 393 placement decisions, so most nodes were selected once or never, and no bandit method can learn from a single observation per option. The limitation is in the event volume the trace offers, not in the update rule.

The difficulty this created was that my own contribution, the discount factor, could not be evaluated at all on data where nothing reorders. I addressed it by constructing a controlled test that produces the condition discounting exists for, reversing the quality ranking of six options midway through and measuring recovery across fifty repetitions. The discounted policy won every repetition in the recovery window. I have been careful in Section 7.4 to state the narrow claim this supports: the mechanism works under reordering, and the trace studied does not reorder. I also learned to be explicit about the reward approximation, since outcomes for nodes not chosen are estimated rather than observed.

Name of student: Malalpola M.L.H.R. (214129X)

My responsibility was the adaptive control component, which adjusts the alert threshold that drives scaling, and the guard that restrains it when it becomes unstable. I implemented the proportional-integral controller, wired adaptive conformal inference to the residual stream produced by the risk model, added the reversal-driven inflation of the permitted adjustment band, and ran the coverage, step-response and instability

assessments in Section 7.5. I also built the actuator described in Section 6.8, without which none of the components would have changed a replica count.

The most valuable thing I learned came from a defect rather than a success. My controller passed its own tests: it responded to a step input, settled in two steps, stayed bounded, and its calibrated coverage was within a point of target. When the three components were first run together over the full trace, the threshold rose during the calm opening, fell during the bursty middle, and then failed to recover. The cause was a missing anti-windup provision, and the reason my own tests never revealed it is that a single step change never asks the controller to reverse twice. A component test that exercises one transition cannot find a fault that requires two.

The difficulty I had to handle honestly concerns my own contribution. On the real bursty segment, adding the reversal guard produced no reduction in reversal count, because that segment contains only four reversals in total, which is too few for any difference to appear. I built a repeated test with fifteen bursts per run over fifty runs, where the mean reversal count fell from 14.04 to 7.64 and the guard was no worse than the alternative in 96 per cent of runs. I also stated a qualification that weakens my own claim rather than waiting to be asked: inflating a conformal interval this way trades a strict coverage guarantee for a practical safety margin, which is a real cost and is recorded in Section 3.4.

Appendix B

Preparation of the Trace Data

This appendix records the procedure summarised in Section 6.3, in the order it was carried out.

A candidate service is selected by counting calls received per service across the call-record tables and cross-checking against the number of container instances each service runs, then choosing one that ranks highly on both. A service with high call volume but a single instance offers nothing to the placement component; a service with many instances but sparse traffic yields unstable percentile estimates.

Response-time percentiles are computed by restricting call records to those where the target service is the receiver, keeping only records taken on the receiving side, taking absolute durations, grouping into two-minute intervals, and computing the ninety-fifth and ninety-ninth percentiles together with the call count per interval. Inbound load is obtained by reshaping the per-service metrics table from its long form into columns and retaining only provider-side entries. Utilisation is obtained by filtering the container table to the chosen service and averaging processor and memory utilisation across its instances per interval, while also recording how many distinct instances were active.

The three resulting tables are joined on the interval index and sorted. Gaps are carried forward for a maximum of two intervals, after which rows lacking a latency reading are dropped. Rates of change over one, two and four intervals are computed for every measurement other than the two counts. The violation threshold is taken as the ninetieth percentile of the observed ninety-ninth-percentile series, the current-interval violation flag is derived from it, and the training label is that flag shifted back by one interval so that each row's features precede the outcome they are paired with. The table is split three-quarters to one-quarter by time position, never at random.

Three checks are applied before the table is used. The proportion of positive labels must fall well short of both extremes, since a value near zero or one indicates a degenerate threshold. Missing-value counts are inspected after the gap-filling step. And where the latency series shows a visible excursion, the label is confirmed to turn positive around it, which is the single most convincing indication that the pipeline behaves as intended.

Appendix C

Selected Implementation Extracts

The extracts below correspond to the algorithms in Chapter 6. The complete source is held in the project repository.

Discounted belief update (placement component). The discount is applied to the accumulated counts before the reward is added, which decays older evidence towards the uninformative prior:

```
self.alpha[node_id] = 1.0 + self.gamma * (self.alpha[node_id] -
1.0) + reward
self.beta[node_id]  = 1.0 + self.gamma * (self.beta[node_id]  -
1.0) + (1.0 - reward)
```

Listing C.1: Posterior update with discounting, from the placement component

Anti-windup condition (control component). Integral accumulation is suppressed only when the output already sits at a bound and the present error would drive it further past that bound, so a correction that cannot be applied is never stored:

```
at_upper = output >= self.upper_bound - EPSILON
at_lower = output <= self.lower_bound + EPSILON
pushing_out = (at_upper and error > 0) or (at_lower and error <
0)
if not pushing_out:
    self.integral += error
```

Listing C.2: Anti-windup condition, from the control component

Band rule (actuator). The asymmetric lower trigger provides hysteresis, so a signal hovering near the threshold does not produce repeated reversals:

```
if signal > threshold and replicas < MAX_REPLICAS and not
cooling_down:
    target = replicas + 1
elif signal < threshold * 0.5 and replicas > MIN_REPLICAS and
not cooling_down:
    target = replicas - 1
else:
    target = replicas
```

Listing C.3: Replica decision rule, from the actuator

Trial count assertion (analysis script). The analysis refuses to proceed unless every configuration contributes exactly five trials, so a partially completed comparison cannot be reported as a complete one:

```
assert all(n_per_arm.get(arm, 0) == TRIALS_PER_ARM for arm in
ARMS), \
    f"expected {TRIALS_PER_ARM} trials per arm, found
{n_per_arm}"
```

Listing C.4: Trial-count assertion, from the analysis script